VECTOR

MICROSAR Classic RTE Analyzer

Technical Reference

Version 4.32.0

Authors	visso, visgme
Status	Released

Document Information

History

Author	Date	Version	Remarks
visso	2015-09-25	0.5	Initial creation for RTE Analyzer 0.5.0
visso	2016-02-26	0.6	Update for RTE Analyzer 0.6.0
visso	2016-07-07	0.7	Described Configuration Feedback and Template Variant Check
visso	2016-10-20	0.8	Configuration Feedback extensions
visso viscpi	2017-03-23	0.9	Updated for RTE Analyzer 0.9.0 Removed BETA disclaimer Fixed chapter numbering
visso	2017-05-09	1.0	Update for RTE Analyzer 1.0.0
visso	2018-03-12	1.1	Update for RTE Analyzer 1.1.0
visso	2018-08-08	1.2	Update for RTE Analyzer 1.2.0
visso	2018-10-26	1.3	Update for RTE Analyzer 1.3.0
visso vislsa	2019-05-10	1.4	Update for RTE Analyzer 1.4.0 Minor improvements
visso	2019-08-28	1.5	Updated for RTE Analyzer 1.5.0
visso	2020-04-06	1.6	Updated for RTE Analyzer 1.6.0
visso	2020-09-21	1.7	Updated for RTE Analyzer 1.7.0 Adapted to new template
visgme visso	2021-04-08	1.8	Updated for RTE Analyzer 1.8.0 Added CANoeApi.h to static files Minor improvements Incremented Perl Version
visso	2021-11-18	1.9	Updated for RTE Analyzer 1.9.0
visso	2021-11-18	1.10	Updated for RTE Analyzer 1.10.0
visso	2022-03-02	1.11	Updated for RTE Analyzer 1.11.0
visso	2022-07-14	1.12	Updated for RTE Analyzer 1.12.0
visso	2022-07-14	1.13	Updated for RTE Analyzer 1.13.0 Added option to extract user types
visso	2022-10-17	4.32	Added RTM stub Minor improvements

Reference Documents

No.	Source	Title	Version
[1]	ISO	ISO/IEC 9899:1990, Programming languages -C	Second edition
[2]	AUTOSAR	Specification of RTE	R4.2.2

Scope of the Document

This technical reference describes the general use of the MICROSAR Classic RTE Analyzer static code analysis tool. This document is relevant for developers that want to integrate a generated RTE into an ECU with functional safety requirements. All aspects that concern the generation of the RTE are described in the technical reference of the RTE.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2023 Vector Informatik GmbH

Contents

1 Introduction 6

2 Functional Description 7

3 RTE Analysis and Integration 9
3.1 Scope of Delivery 9
3.1.1 Static Files 9
3.1.2 Dynamic Files 13
3.2 Restrictions 13
3.3 RTE Analyzer Command Line Options 14
3.4 Analysis Report Contents 15
3.4.1 Analyzed Files 15
3.4.2 Configuration Parameters 15
3.4.3 Findings 17
3.4.4 Configuration Feedback 24
3.4.5 Template Variant Check 27
3.5 Integration into DaVinci CFG 28

4 Glossary and Abbreviations 31
4.1 Glossary 31
4.2 Abbreviations 31

5 Additional Copyrights 32

6 Contact 33

Tables

Table 2-1 Supported features 8
Table 3-1 Static files 12
Table 3-2 Assumed platform type sizes 12
Table 3-3 Generated files 13
Table 3-4 RTE Analyzer Command Line Options 14
Table 3-5 Analysis parameters that are extracted from the configuration 16
Table 3-6 RTE Analyzer Findings 24
Table 4-1 Glossary 31
Table 4-2 Abbreviations 31
Table 5-1 Free and Open Source Software Licenses 32

Figures

Figure 3-1 Project menu 28
Figure 3-2 External generation steps 29
Figure 3-3 Code generation 30

1 Introduction

This document describes the static code analysis tool MICROSAR Classic RTE Analyzer. The MICROSAR Classic RTE Analyzer is part of MICROSAR Classic Safe RTE. MICROSAR Classic Safe RTE provides an AUTOSAR RTE generator that is developed with an ISO26262 compliant development process, to allow the usage of the generated RTE code within an ECU with functional safety requirements.

The MICROSAR Classic RTE Analyzer analyzes the generated RTE code for errors with a special emphasis on sporadic runtime errors that are hard to detect during ECU integration tests.

2 Functional Description

The features listed in the following table cover the complete functionality of the MICROSAR Classic RTE Analyzer.

Supported Features
Compilation check for RTE code
Detection of disallowed inline assembly usage in RTE APIs
Detection of template variants that are not allowed for SafeRTE
Detection of template combinations that are not allowed for SafeRTE
Detection of accesses to invalid pointers in RTE APIs
Detection of out of bounds write accesses in RTE APIs
Detection of memcpy operations with overlapping pointers in RTE APIs
Detection of global RTE variables that are not initialized
Detection of interrupt lock API sequence mismatches in RTE APIs
Detection of OS APIs that are wrongly called with locked interrupts in RTE APIs
Detection of data consistency APIs that are called from the wrong context in RTE APIs
Detection of RTE variables that are accessed from concurrent execution contexts without protection
Detection of RTE variables that are accessed from multiple cores and that are not mapped to non-cacheable memory sections
Detection of concurrent calls to non-reentrant APIs within RTE APIs
Detection of prohibited write accesses in non-trusted partitions
Configuration Feedback for scheduling properties
Configuration Feedback for executable entities
Configuration Feedback for unreachable RTE APIs and entities
Configuration Feedback for RTE APIs that require a valid COM buffer configuration
Configuration Feedback for RTE APIs that require a valid NVM buffer configuration
Automatic verification of COM buffer assumptions for MICROSAR Classic COM
Automatic verification of NVM buffer assumptions for MICROSAR Classic NVM
Configuration Feedback for non-typesafe interfaces to the BSW and SWCs where a call with a wrong parameter might cause out of bounds writes by the RTE or a called runnable/BSW API
Configuration Feedback for RTE APIs for which a call from a wrong context might cause data consistency problems
Configuration Feedback for RTE APIs that are blocking
Configuration Feedback for RTE APIs that communicate with other ECUs
Configuration Feedback for RTE APIs with queues
Configuration Feedback for RTE APIs with serializer transformers
Configuration Feedback for RTE APIs with alive timeout handling

Supported Features
Configuration Feedback for RTE APIs with invalidation handling
Configuration Feedback for RTE APIs with never received handling
Configuration Feedback for RTE APIs with initial value handling
Configuration Feedback for RTE APIs with E2E transformer handling
Configuration Feedback for RTE APIs with data conversion
Configuration Feedback for RTE APIs that access non-volatile memory
Configuration Feedback for exclusive areas
Configuration Feedback for connections
Configuration Feedback for recursive calls
Configuration Feedback for spinlocks that need to protect from task interruptions
Configuration Feedback for RTE Implementation Plugins
RTE Analyzer Configuration generation by DaVinci CFG
Analysis report generation
Configuration Feedback Generation for QM and ASIL partitions
Detection of COM and LDCOM calls in the wrong partition
Extraction of user types

Table 2-1 Supported features

3 RTE Analysis and Integration

This chapter gives necessary information about the content of the delivery, the usage of the MICROSAR Classic RTE Analyzer and a description of the generated report.

3.1 Scope of Delivery

The delivery contains the files which are described in the chapters 3.1.1 and 3.1.2:

3.1.1 Static Files

File Name	Description
MicrosarRteAnalyzer.exe	MICROSAR Classic RTE Analyzer command line frontend
MicrosarRteAnalyzerCfgGen.exe	MICROSAR Classic RTE Analyzer configuration file generator (automatically invoked by DaVinci CFG during RTE generation)
MicrosarRteAnalyzerCfgGen64.exe	MICROSAR Classic RTE Analyzer configuration file generator for 64-bit windows (automatically invoked by Davinci CFG during RTE generation)
MicrosarRteAnalyzer	MICROSAR Classic RTE Analyzer linux command line frontend (x86_64, glibc 2.26)
MicrosarRteAnalyzerCfgGen	MICROSAR Classic RTE Analyzer configuration file generator linux command line frontend (x86_64, glibc 2.26) (automatically invoked by DaVinci CFG during RTE generation)
MicrosarRteAnalyzerCfgGen64	MICROSAR Classic RTE Analyzer configuration file generator linux command line frontend (x86_64, glibc 2.26) (automatically invoked by DaVinci CFG during RTE generation)
Settings_RteAnalyzer.xml	DaVinci CFG adaption module
TechnicalReference_RteAnalyzer.pdf	This document
MicrosarIRAnalyzer.exe	Analysis backend (used internally by MicrosarRteAnalyzer.exe)
License_Artistic.txt	Perl license
License_LLVM.txt	LLVM/CLANG license
Com.h	Stub Com header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Compiler.h	Stub Compiler header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Compiler_Cfg.h	Stub Compiler_Cfg header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
ComStack_Cfg.h	Stub ComStack_Cfg header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)

File Name	Description
ComStack_Types.h	Stub ComStack_Types header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Det.h	Stub Det header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E.h	Stub E2E header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P01.h	Stub E2E Profile 1 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P02.h	Stub E2E Profile 2 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P04.h	Stub E2E Profile 4 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P05.h	Stub E2E Profile 5 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P06.h	Stub E2E Profile 6 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P07.h	Stub E2E Profile 7 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P08.h	Stub E2E Profile 8 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P11.h	Stub E2E Profile 11 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P22.h	Stub E2E Profile 22 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_P22.h	Stub E2E Profile 44 header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
E2E_SM.h	Stub E2E State Machine header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
EcuM_Error.h	Stub ECUM Error header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Float.h	Stub Float header (used internally by

File Name	Description
	MicrosarRteAnalyzer.exe for standalone RTE verification)
Ioc.h	Stub Ioc header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
LdCom.h	Stub LdCom header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
MemMap.h	Stub MemMap header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
MemMap_Common.h	Stub MemMap header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
NvM.h	Stub NvM header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Os.h	Stub Os header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Os_MemMap.h	Stub Os_MemMap header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Os_MemMap_OsCode.h	Stub Os_MemMap header for generation 7 OS (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Platform_Types.h	Stub Platform_Types header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Rtm.h	Stub Rtm header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Std_Types.h	Stub Std_Types header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
String.h	Stub String header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Xcp.h	Stub Xcp header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
XcpProf.h	Stub XcpProf header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)
Vstdlib.h	Stub VStdlib header (used internally by MicrosarRteAnalyzer.exe to extract the COM signal

File Name	Description
	lengths)
CANoeApi.h	Stub CANoeApi header (used internally by MicrosarRteAnalyzer.exe for standalone RTE verification)

Table 3-1 Static files

The RTE Analyzer assumes the following maximum data type sizes:

Type	Size in Bits
sint8	8
uint8	8
sint16	16
uint16	16
sint32	32
uint32	32
sint64	64
uint64	64
float32	32
float64	64
enum	32

Table 3-2 Assumed platform type sizes



Caution
If the sizes of the platform types exceed the sizes described in Table 3-2, contact Vector as the size of the data types is used for the data consistency checks.

3.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci CFG to the RteAnalyzer subdirectory when the RTE is generated.

File Name	Description
RteAnalyzerConfiguration.json	Configuration file for MICROSAR Classic RTE Analyzer.
<BSW>.c	These files contain stub implementations for the schedulable entities that call all available RTE APIs.
TestControl.c	This file contains stubs for the BSW calls to the RTE.
<SWC>.c	These files contain stub implementations for the runnables that call all available RTE APIs.
Com_Cfg.h	This file contains the configuration for the stub COM module.
LdCom_Cfg.h	This file contains the configuration for the stub LDCOM module.
Os_Cfg.h	This file contains the configuration for the stub OS module.
Ioc_Cfg.h	This file contains the IOC configuration for the stub OS module.
Xcp_Cfg.h	This file contains the configuration for the stub XCP module.
NvM_Cfg.h	This file contains the configuration for the stub NVM module.
E2EXf_Cfg.h	This file contains the configuration for the stub E2EXf module.
Rte_Rips_<rip>.h	This file contains the prototypes for the RIP plugins.

Table 3-3 Generated files



Caution
When the BSW Package is moved, the RTE needs to be regenerated to update the RTE Analyzer configuration file as some include paths point to the BSW Package.

Besides the files that are generated by the DaVinci CFG, the RTE Analyzer generates the following files when invoked from the command line.

File Name	Description
AnalysisReport.txt	Report that contains the results of the static code analysis and analysis assumptions that need to be reviewed by the user of the MICROSAR Classic RTE Analyzer.

3.2 Restrictions

The MICROSAR Classic RTE Analyzer uses a Compiler front end in order to compile the input source files. This Compiler front end requires ANSI-C 90 [1] conform source code. Some target compilers implement specific language extensions which might prevent the MICROSAR RTE Analyzer from compiling the code successfully. The Vector BSW code

does not contain such language extensions. However, these extensions may be included via customer header files. In such a case the customer shall take care that these language extensions are encapsulated via the preprocessor for the MICROSAR Classic RTE Analyzer execution. The corresponding preprocessor switches can be specified via the command line when calling the MICROSAR Classic RTE Analyzer.

3.3 RTE Analyzer Command Line Options

The frontend RTE Analyzer starts the static code analysis. It can be started on the command line once the RTE and the MICROSAR Classic RTE Analyzer configuration were generated by the DaVinci CFG.

Option	Description
`-c <config>`	Selects the configuration file of the project that shall be analyzed.
`-I <dir>`	Add directory name <dir> to include file search path
`-D <name>[=<value>]`	Defines macro with name <name> and value <value>
`-o <path>`	Selects the directory to which the analysis report will be written
`-e`	Extended Configuration Feedback. If not set, the Configuration Feedback will not include RTE functionality in OS Applications with SafetyLevel QM or OS Applications for which no SafetyLevel is configured in the ECUC module.
`-d`	Disable analysis of the COM and the NVM generation data
`-j`	Number of threads for compilation. Defaults to the number of processors.
`-V`	Shows the version
`-h`	Shows the command line help
`-w`	Windows version (64Bit or 32Bit)
`-u`	Store a preprocessed version of Rte_UserTypes.h in the analysis report directory. This header can be provided to Vector together with the support request package to allow the analysis without all the customer specific headers that might be included by Rte_UserTypes.h.

Table 3-4 RTE Analyzer Command Line Options

Example:
```
MicrosarRteAnalyzer.exe -c RteAnalyzerConfiguration.json –o
Reports
```



Note
The compilation fails when the GenData directory contains headers from other modules that include headers that are not contained in the include search path. Please run the analysis for a GenData folder that only contains the RTE and transformer sources then.

3.4 Analysis Report Contents

The RTE Analyzer prints errors that prevent the analysis of the system to the console.

When the RTE Analyzer was executed without errors, an analysis report is written to the output directory that contains potential problems within the generated RTE.

These problems are only listed in the report and not printed to the console.

As not every detected violation necessarily leads to an error in the ECU, the final decision whether an issue is critical or not is up to the user of the RTE Analyzer.

Besides the detected constraint violations, the analysis report also contains assumptions about the system that was derived from the configuration.

These assumptions need to be verified by the user of the RTE Analyzer.

3.4.1 Analyzed Files

The report starts with the version of the analysis report, the time of the analysis and the name of the windows user that initiated the analysis.

Moreover, the analyzed files are listed. It needs to be assured that the correct files were analyzed and no file is missing.

3.4.2 Configuration Parameters

The MICROSAR Classic RTE Analyzer relies on configuration parameters from the DaVinci CFG to determine the scheduling properties of the individual tasks and BSW callbacks.

These parameters need to be reviewed because a wrong parameter might lead to missed data consistency problems.

The report contains the following parameters that need to be checked against the target system.

Parameter	Description
MaxAtomicMemoryAccess	Describes the maximum number of bytes for variable accesses up to which the compiler will emit an atomic access instruction.

Parameter	Description
BswOsApplication	Describes the OS Application from which the RTE Communication Callbacks (`Rte_COMCbk, Rte_LdCom, …`) are called.
NvMBswOsApplication	Describes the OS Application from which the RTE NVM Callbacks (`Rte_GetMirror, Rte_SetMirrors, …`) are called.
OsApplications	Lists the OS Applications in the system
OsApplicationName	Name of the OS Application
CoreId	ID of the Core that contains the OS Application
IsTrusted	Describes if the OS Application runs without MPU (IsTrusted == 1) or with MPU (IsTrusted == 0). Please note that trusted applications with protection are also listed with IsTrusted == 0.
IsPrivileged	Describes if the OS Application runs in supervisor mode (IsPrivileged == 1) or in user mode (IsPrivileged == 0)
SafetyLevel	SafetyLevel of the OS Application: QM, ASIL_A, ASIL_B, ASIL_C, ASIL_D
Tasks	List of OS Tasks that are assigned to the OS Application
TaskName	Name of the OS Task
Priority	Priority of the OS Task
Preemption	Preemption setting of the OS Task
Callbacks	List of callbacks that are assigned to the OS Application
CallbackName	Name of the callback
ExecutedOnTaskLevel	Describes if the callback is called from a task (1) or from an ISR (0).
SignalsWithoutCallback	List of signals without callback that are assigned to the OS Application

Table 3-5 Analysis parameters that are extracted from the configuration

3.4.3 Findings

The RTE Analyzer currently reports the findings described in Table 3-6. The description describes the possible findings in more detail and the actions that need to be taken when they are contained in the analysis report.

ID	Headline	Description
11000	Unsupported integer to pointer conversion	The RTE code uses an integer value that was casted to a pointer type. Example: *uint8* ptr = 0xdeadbeef;* **ptr = 5;* This code construct must not be used in the RTE code. Contact Vector.
11001	Unsupported inline assembly	The RTE code uses inline assembly. Example: *asm("add %al, (%rax)");* This code construct must not be used in the RTE code. Contact Vector.
11006	Unsupported path to pointer target	The pointer analysis detected a code construct that it cannot handle. This code construct must not be used in the RTE code. Contact Vector.
11007	Unsupported usage of memcpy or memset	Memcpy or memset is used through an indirect function call. This code construct must not be used in the RTE code. Contact Vector.
11009	Unsupported instruction	An instruction was encountered. This code construct must not be used in the RTE code. Contact Vector.
11010	Division by zero	Division by a constant zero value was detected. This leads to runtime exceptions and must not be used in RTE code. Contact Vector.
12000	Potential out of bounds write	A pointer that was already used in the preparation of the analysis is outside of the assumptions that were used during the preparations. Example: *typedef struct {*

ID	Headline	Description
		uint8 a;* *uint8* b;* *} struct_t;* *struct_t s;* *uint8** ptr = &s.a;* *ptr[1][0] = 7;* This code construct must not be used in the RTE code. Contact Vector.
12001	Potential null pointer write	An RTE API writes to a pointer that may be null. This code construct must not be used in the RTE code. Contact Vector.
12002	Potential out of bounds write	An RTE API writes outside of the bounds of a variable. Example: *uint8 a[5];* *a[5] = 1;* This code construct must not be used in the RTE code. Contact Vector.
12003	Potential access of invalid pointer	An RTE API accesses an invalid pointer. This code construct must not be used in the RTE code. Contact Vector.
13000	Unexpected lock sequence	A lock function is not followed by an appropriate unlock function. Example: *SuspendAllInterrupts();* *a = 5;* *ResumeOSInterrupts();* This code construct must not be used in the RTE code. Contact Vector.
13001	Different lock states for loop	A function uses different lock states in different loop iterations. Example: *for (i = 0; i < 20; i++)* *{* *if (i == 5)*

ID	Headline	Description
		{ *DisableAllInterupts();* *}* *if (i == 6)* *{* *EnableAllInterrupts();* *}* *}* This code construct must not be used in the RTE code. Contact Vector.
13002	Different lock states for call	A call may be done with and without prior locking. Example: *if (a == 0)* *{* *DisableAllInterrupts();* *}* *Function();* This code construct must not be used in the RTE code. Contact Vector.
13003	Different lock states for recursive call	A recursive function changes the lock state prior to the next recursion. Example: *void func()* *{* *DisableAllInterrupts();* *func();* *EnableAllInterrupts();* *}* This code construct must not be used in the RTE code. Contact Vector.
13004	Different lock states for return	A function may return a with different lock state. Example: *DisableAllInterrupts();*

ID	Headline	Description
		if (a) *{* *return;* *}* *EnableAllInterrupts();* *return;* This code construct must not be used in the RTE code. Contact Vector.
13005	Task or ISR returns with locked interrupts	An RTE Task or callback returns with locked interrupts in at least one code branch. This code construct must not be used in the RTE code. Contact Vector.
13006	OS API called with locked interrupt	An OS API e.g. `WaitEvent` is called with locked interrupts. This is prohibited by the OS specification. This code construct must not be used in the RTE code. Contact Vector.
13007	OS API called with disabled interrupts	An OS API e.g. `SuspendOSInterrupts` is called within a section that is locked with DisableAllInterrupts. This is prohibited by the OS specification. This code construct must not be used in the RTE code. Contact Vector.
13008	OS API called in wrong context	An optimized MICROSAR Classic interrupt lock API is called from the wrong context. E.g. an optimized lock API for trusted OS applications is called from an untrusted application. This code construct must not be used in the RTE code. Contact Vector.
13009	Accesses can interrupt each other	The RTE Analyzer detected that a variable is accessed from multiple tasks that can interrupt each other. The variable is not protected by an OS API e.g. `interrupt lock or spinlock.` This code construct must not be used in the RTE code. Contact Vector. Please note that RTE Analyzer has no knowledge about additional RteExclusiveAccessOptimization settings in the configuration, therefore such accesses are also listed in this category although the accesses are exclusive in the actual target system.
13010	Non-reentrant function with non-constant	The RTE Analyzer checks the RTE for concurrent calls to BSW APIs. If the

ID	Headline	Description
	handle	reentrancy depends on the handle, the handle needs to be constant so that it can be analyzed by the RTE Analyzer. This code construct must not be used in the RTE code. Contact Vector.
13011	Non-reentrant function invoked concurrently	The RTE Analyzer detects concurrently called functions for which the caller would have needed to assure non-reentrant calls. This code construct must not be used in the RTE code. Contact Vector.
13012	Different resources used on same core	The RTE code uses different resources to protect the same variable. If a variable needs to be protected from concurrent accesses in multiple tasks, the same resource needs to be used for all accesses. This code construct must not be used in the RTE code. Contact Vector.
13013	Different spinlocks used	The RTE code uses different spinlocks to protect the same variable on a single core. If a variable needs to be protected from concurrent accesses on multiple cores, the same spinlock needs to be used for all accesses. This code construct must not be used in the RTE code. Contact Vector.
13014	Not all accesses protected with resource	The RTE code does not always use resources to protect a variable. If a variable needs to be protected from concurrent accesses in multiple tasks, the same resource needs to be used for all accesses. This code construct must not be used in the RTE code. Contact Vector.
13015	Bitfield write access without interrupt locks	The RTE uses interrupt locks to prevent read modify write problems in bitfields. The RTE Analyzer detected an access without locks. This code construct must not be used in the RTE code. Contact Vector.
13017	Lock type cannot be nested	The RTE uses a non-nestable interrupt lock nestedly. This code construct must not be used in the RTE code. Contact Vector.
13018	Lock uses unsupported handle parameter	The RTE uses a nonconstant handle for spinlocks or resources. This code construct must not be used in the RTE code. Contact Vector.
14000	Unmatched memory section	A memory section was not closed correctly. Example:

ID	Headline	Description
		#define RTE_START_SEC_VAR *#include “MemMap.h”* *uint8 var;* *#define RTE_STOP_SEC_CONST* *#include “MemMap.h”* This code construct must not be used in the RTE code. Contact Vector.
14001	Variable not mapped to memory section	A variable is declared without being mapped to a memory section. Example: *uint8 var;* *#define RTE_START_SEC_VAR* *#include “MemMap.h”* This code construct must not be used in the RTE code. Contact Vector.
14002	Variable not mapped to NOCACHE memory section	A variable that is accessed from multiple cores is not mapped to a NOCACHE memory section. This may lead to data consistency problems. Example: *#define RTE_START_SEC_VAR* *#include “MemMap.h”* *uint8 var;* *#define RTE_STOP_SEC_VAR* *#include “MemMap.h”* This code construct must not be used in the RTE code. Contact Vector.
16000	Missing task info	The configuration contains no task settings for the task. Possible reason: a function ends with the name func and is missdetected as OS Task by the RTE Analyzer. Rename the function or ignore the message.
17000	Potential illegal memcpy	Memcpy is called with overlapping source and destination arguments. Example:

ID	Headline	Description
		Rte_MemCpy(dst, dst, 5); This code construct must not be used in the RTE code. Contact Vector.
17001	Potential illegal memcpy (undeterminable)	Memcpy may be called with overlapping source and destination arguments. The size could not be resolved. This code construct must not be used in the RTE code. Contact Vector.
20000	Use of uninitialized variables	The RTE code reads variables that are not initialized in `Rte_Start` or `Rte_InitMemory`. This code construct must not be used in the RTE code. Contact Vector.
20001	Inconsistent handling of never received flags	The RTE code accesses never received flags that are not set on the sending side. This code construct must not be used in the RTE code. Contact Vector.
20002	Inconsistent handling of invalidate flags	The RTE code accesses invalidate flags that are not set on the sending side. This code construct must not be used in the RTE code. Contact Vector.
20003	Inconsistent handling of update flags	The RTE code accesses invalidate flags that are not set on the sending side. This code construct must not be used in the RTE code. Contact Vector.
20004	Inconsistent handling of idle flags	The RTE code accesses idle flags in an unsupported way. This code construct must not be used in the RTE code. Contact Vector.
20005	Inconsistent handling of overflow flags	The RTE code accesses overflow flags in an unsupported way. This code construct must not be used in the RTE code. Contact Vector.
20006	Inconsistent handling of call completed flags	The RTE code accesses call completed flags in an unsupported way. This code construct must not be used in the RTE code. Contact Vector.
20007	Missing Schedule for synchronous client-server call	The RTE code synchronously calls a server runnable on another task without Schedule and WaitEvent but the client task is not preemptive. This code construct must not be used in the RTE code. Contact Vector.
20008	Missing WaitEvent for synchronous client-server call	The RTE code synchronously calls a server runnable on another task without

ID	Headline	Description
		WaitEvent but the server task does not have a higher priority. This code construct must not be used in the RTE code. Contact Vector.
20009	Missing overflow flags	The RTE code uses an RTE IOC implementation without overflow flag handling for queued sender-receiver communication. This code construct must not be used in the RTE code. Contact Vector.
20010	Inconsistent memory mapping	The RTE code in a non-trusted partition contains a write access to a variable from another non-trusted partition. Such writes lead to MPU violations during runtime and must not be used in RTE code. Contact Vector.
20011	COM or LDCOM API called from wrong partition	The RTE code calls a COM and LDCOM API in one partition for a signal that is configured for a different partition. Such calls are not allowed by the called modules and must not be used in RTE code. Contact Vector.

Table 3-6 RTE Analyzer Findings

3.4.4 Configuration Feedback

The findings from chapter 3.4.3 describe inconsistencies within the generated RTE. However, also a consistently generated RTE may violate functional safety requirements when the generated RTE does not match the intentions of the user e.g. when wrong configuration parameters were chosen for the intended use case.

Therefore, during development of the RTE Generator a safety analysis is performed on all input parameters of the generator in order to detect functionality for which a slightly different configuration leads to the generation of APIs with compatible C signature but different runtime behavior.

The RTE Analyzer lists the detected functionality in the analysis report, so that an integration test as required by ISO26262 can confirm that only the intended and no unintended functionality is implemented in the generated RTE.

This also makes it possible to use the MICROSAR Classic RTE Generator in combination with non-TCL1 configuration tools as unintended configuration modifications by the tool will lead to an unexpected configuration feedback.

By default, the configuration feedback is only printed for the OS Applications with ASIL safety levels. When the –e configuration switch is enabled, the RTE functionality in OS Applications with SafetyLevel QM is also included. Analysis report contains the following information:

- Function may be called recursively - The software design contains e.g. configured client server calls that may lead to recursive calls. ISO26262 recommends that recursion is not used in the software design and implementation.
- Uncalled function - A function e.g. a server runnable without connected client was encountered during the analysis. Functions that are not called are not analyzed by the RTE Analyzer. Assure that the function is not called in the target system, either.
- Call with non-typesafe parameters - Some APIs contain pointers that are not typesafe e.g. because the parameter type is a pointer to the base type and the function writes more than a single element of this type. The parameter may also be a void pointer type. The RTE Analyzer lists these functions so that it can be verified that the passed buffer matches the expectations of the called function. Please note that the buffer that is listed by the RTE Analyzer might be larger than the actual number of bytes that are written by the called function.
- COM call with non-typesafe parameters – The COM APIs for data reception are not typesafe. It has to be assured, that the COM does not write more bytes than expected by the RTE. If MICROSAR Classic COM is used, the RTE Analyzer extracts the number of written bytes from the generated COM sources. When the message contains (ok) the number of bytes could automatically be checked in the generation data of the COM module. When the message contains (not ok), the buffer in the generation data of the COM module is assumed to be too small. The message lists the signal sizes for all variants, not only the variants in which a signal is used by the RTE what can lead to additional (not ok) entries that need to be verified manually. If the RTE Analyzer cannot compile the COM module due to missing headers or defines, the analysis can be disabled with the -d parameter. In this case, the length assumptions need to be checked manually by reviewing the COM generation data or assuring that the COM API calls are performed in a partition that is separated from the ASIL partition.
- NVM callback with non-typesafe parameters – The NVM GetMirror callback does not have typesafe parameters. It has to be assured that the buffer that is passed by the NVM is not smaller than the number of bytes that are written by the RTE. If MICROSAR Classic NVM is used, the RTE Analyzer extracts the available number of bytes from the generated NVM sources. When the message contains (ok) the number of bytes could automatically be checked in the generation data of the NVM module. When the message contains (not ok), the buffer in the generation data of the NVM module is assumed to be too small.
- API for Safe component must not be called from wrong context - The RTE generator disables task priority optimizations for partitions with an ASIL Safety Level. If an API is used only on a single task according to the configuration or if the parameter OptimizeLocksInASILPartitions is set, the RTE generator optimizes nevertheless. The RTE Analyzer lists these APIs and the QM APIs that are connected to them so that it can be confirmed that the APIs are not accidently called from a runnable for which no port access was configured in the configuration.
- Spinlock needs to provide task protection – The RTE generator does not generate interrupt locks if code is protected by a spinlock that is configured to protect against task interruption on the same core. The RTE Analyzer lists affected spinlocks so that it can be confirmed that the spinlock in the OS is configured correctly. The

spinlock literals for the reported numeric identifiers can be found in RteAnalyzer/Source/Os_Cfg.h when not all spinlocks are configured to lock interrupts.

- Spinlock needs to provide interrupt protection – The RTE generator does not generate interrupt locks if code is protected by a spinlock that is configured to protect against interrupts on the same core. The RTE Analyzer lists affected spinlocks so that it can be confirmed that the spinlock in the OS is configured correctly. The spinlock literals for the reported numeric identifiers can be found in RteAnalyzer/Source/Os_Cfg.h when not all spinlocks are configured to lock interrupts.
- Spinlock needs to provide interrupt protection for bitfields – The RTE generator does not generate interrupt locks if code is protected by a spinlock that is configured to protect against interrupts on the same core. The RTE Analyzer lists affected spinlocks so that it can be confirmed that the spinlock in the OS is configured correctly. The spinlock literals for the reported numeric identifiers can be found in RteAnalyzer/Source/Os_Cfg.h when not all spinlocks are configured to lock interrupts.
- Spinlocks must provide deadlock protection - When multiple cores contain unprotected accesses to spinlocks in different tasks that can interrupt each other, nesting of spinlocks is possible although the RTE itself does not use them nestedly. When one task on a core aquires spinlock A and another task on a different core at the same time aquires spinlock B and one of the tasks gets interrupted by another task that tries to aquire one of the already locked spinlocks a deadlock occurs. The RTE Analyzer reports a configuration feedback entry when it detects unprotected spinlock accesses on different cores so that it can be confirmed that the spinlock in the OS are configured correctly.
- Non-Queued connections – This contains a list of all non-queued intra-ECU sender-receiver connections between `Rte_Write`, `Rte_IWrite`, `Rte_Read`, `Rte_DRead`, `Rte_IRead`. In case the connection involves NV Block SWCs, also other sender and receivers that access the same NV block are listed, even if there is no assembly connector element in the configuration.
- Queued connections – This contains a list of all queued intra-ECU sender-receiver communication connections between `Rte_Send` and `Rte_Receive`.
- Inter-runnable connections – This contains a list of all inter-runnable variable connections.
- External connections – This contains a list of all the APIs and server runnables that communicate with other ECUs.
- Switch-mode connections – This contains a list of all mode connections between `Rte_Switch` and `Rte_Mode`.
- Exclusive areas – This contains a list of all exclusive areas and their implementation methods. This includes explicit and implicit exclusive areas. The implementation methods need to be set according to the requirements of the application.
- Initial values of APIs – This contains a list of all the APIs that return an initial value. The calling runnable needs to handle the initial value. When RteAnalyzer was able

to extract the initial value from the code, the value is also printed. Currently no value can be extracted for implicit connections or when the initial value is provided by COM or the OS.

- Blocking APIs – This contains a list of all APIs that are blocking. These may unexpectedly delay the calling function. It also contains non-blocking APIs with timeout.
- Executable Entities – This contains a list of all the executable entities. The entities are listed together with the tasks in which they are executed.
- APIs with special return values – This contains a list of all the APIs that return special error codes such as RTE_E_MAX_AGE_EXCEEDED, RTE_E_INVALID and RTE_E_NEVER_RECEIVED.
- APIs with queues – This contains the list of APIs with queues along with the queue sizes.
- APIs with serializer transformers – This contains the list of APIs that use a serializer transformer (e.g. COMXF). The transformer needs to be configured correctly.
- APIs with E2E transformers – This contains the list of APIs that read or write data with the help of the E2E transformer. The communication partner needs to handle the converted data.
- Reentrant Executable Entities – This contains a list of all executable entities that are called reentrantly. This is based on the core id, priority and the preemption setting of the tasks in which the entity is executed.
- APIs using data conversion – This contains a list of all the APIs that do data conversion. The communication partner needs to handle the converted data.
- APIs that may use NVM – This contains a list of all Per Instance Memories and sender-receiver APIs that access NV Block SWCs. The NVM module needs to be configured correctly.
- RTE Implementation Plugins – This contains a list of RTE Implementation Plugins (RIPs). RIPs hook deeply into the RTE and therefore currently cannot be used in the safety related part of the RTE.

Please note that the configuration feedback describes the actual properties of the code. This can be different from the configured values, especially if the APIs are generated for unconnected ports.

Example: An unconnected `Rte_Read` API is configured to return RTE_E_NEVER_RECEIVED. According to the RTE specification, the return value is RTE_E_UNCONNECTED independently of the never received handling, therefore the generated API has no code to return RTE_E_NEVER_RECEIVED and the analysis report does not list the API in the “APIs with special return values” section.

The safety manual describes how the configuration feedback can be used for integration testing.

3.4.5 Template Variant Check

The MICROSAR Classic RTE Generator is a template based code generator. During generation, the MICROSAR Classic RTE Generator calculates checksums for the template

sequences that were used to generate the RTE APIs. The delivery of the generator contains a list of checksums that were approved for the usage in an ECU with functional safety requirements.

The MICROSAR Classic RTE Analyzer checks that the template sequences that were used to generate the analyzed RTE are within the allowed sequences.

Please contact Vector if the analysis report lists template variants that are not within the allowed ones.

3.5 Integration into DaVinci CFG

Since the MICROSAR Classic RTE Analyzer checks the consistency of the generated RTE it is convenient to run the MICROSAR Classic RTE Analyzer automatically after the data is generated. To integrate the MICROSAR Classic RTE Analyzer into the DaVinci CFG, an external generation step can be configured.

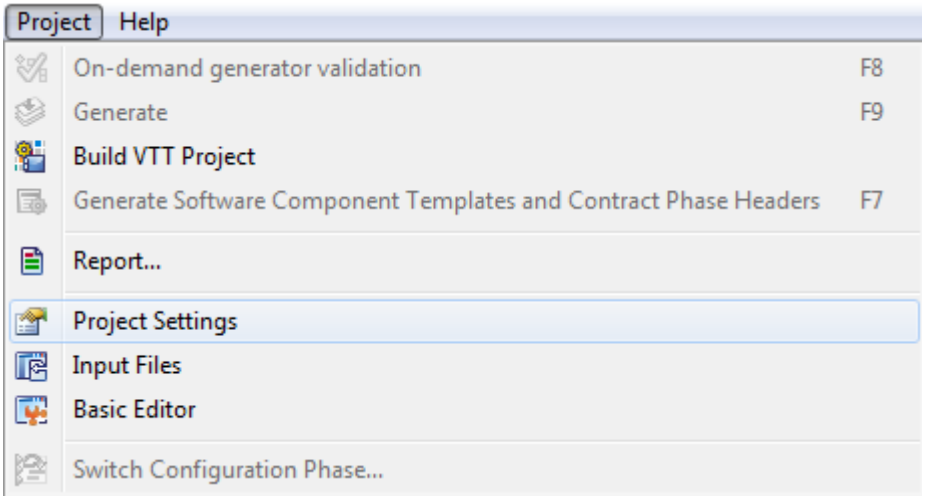


Figure 3-1 Project menu

Start the DaVinci CFG and select the menu “Project”. Next select the menu item “Settings”.

To add a new external generation step, select “External Generation Steps”. This will display the following window:

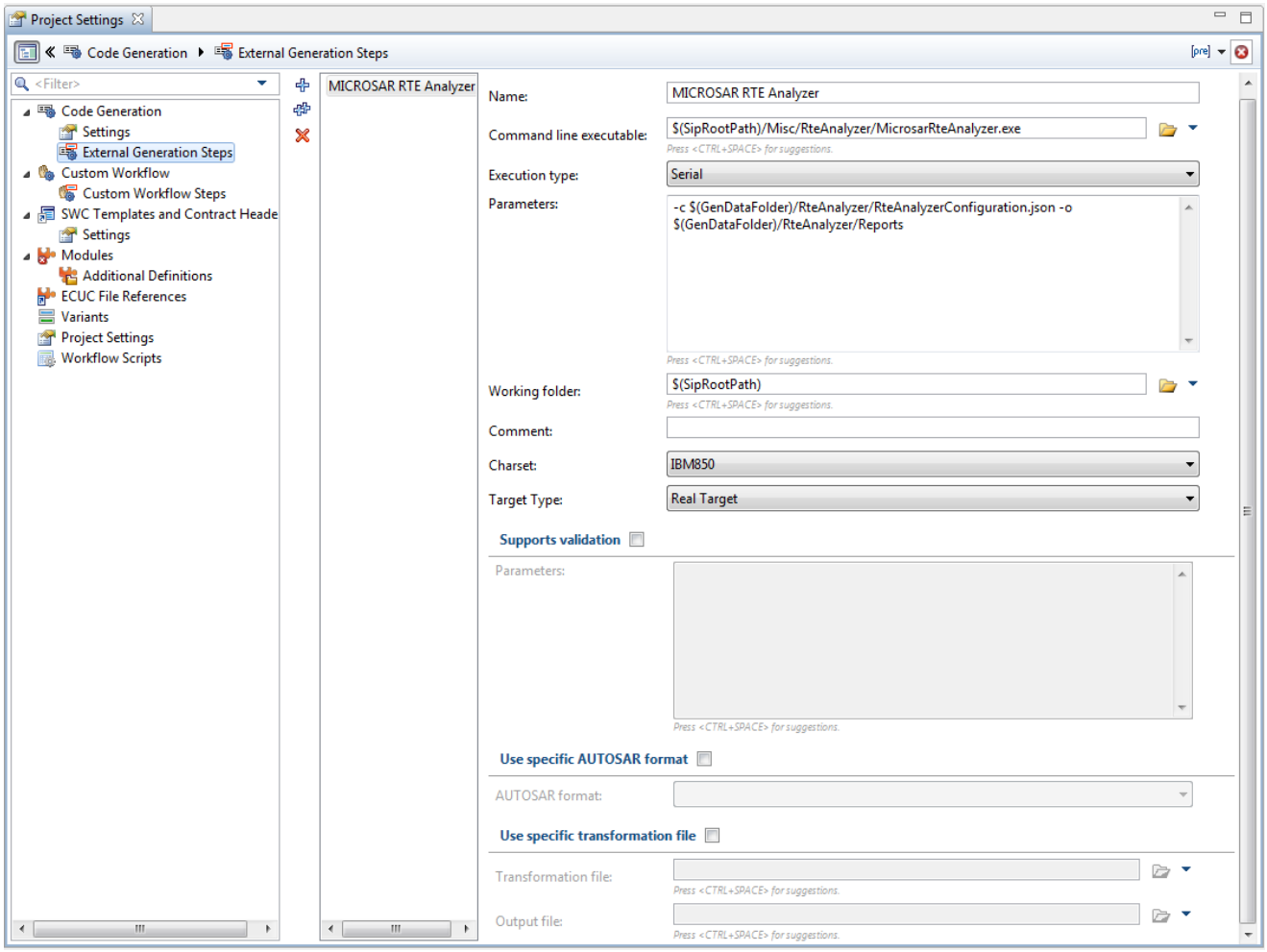


Figure 3-2 External generation steps

Click on the Add button with the “+” symbol and enter the MICROSAR Classic RTE Analyzer path e.g.

```
$(SipRootPath)/Misc/RteAnalyzer/MicrosarRteAnalyzer.exe
```

and command line arguments e.g.

```
-c $(GenDataFolder)/RteAnalyzer/RteAnalyzerConfiguration.json -o
$(GenDataFolder)/RteAnalyzer/Reports
```

For Virtual Target, $(GenDataVTTFolder) needs to be used.

Note
It is required to set a working directory for a post generation step.

Now the external generation step needs to be configured to be run after the DaVinci Generators. To configure this click on the item “Code Generation”.

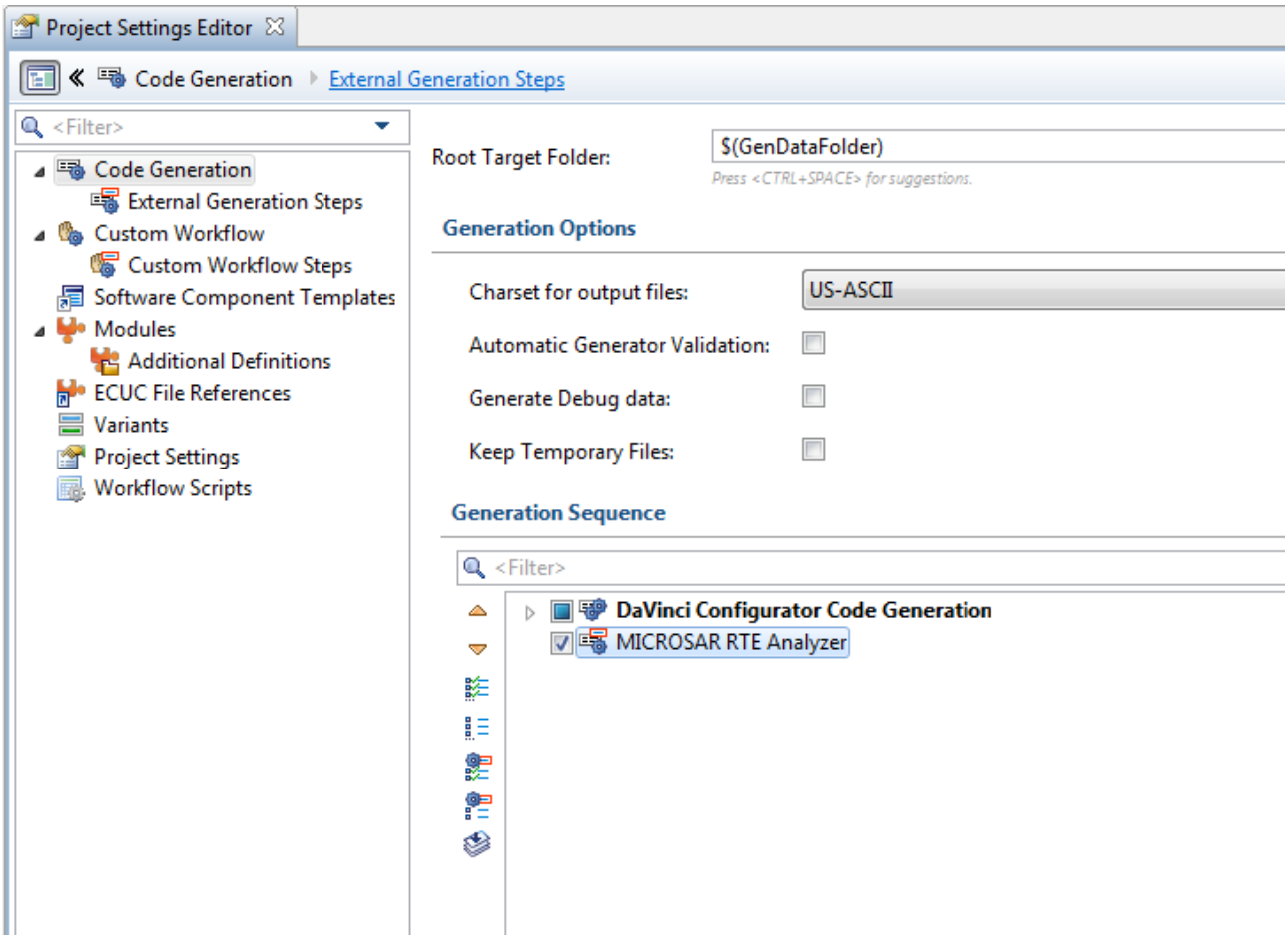


Figure 3-3 Code generation

Now select the MICROSAR Classic RTE Analyzer Generation Step and enable it by checking the check box in front of it. Additionally the MICROSAR Classic RTE Analyzer should be run after the DaVinci CFG generated the data. Therefore it is necessary to move it after the DaVinci Code Generation using the Down button with the “▼” symbol.

Now the MICROSAR Classic RTE Analyzer will be automatically executed after the DaVinci CFG has generated the data.

Note
The MICROSAR Classic RTE Analyzer will also be executed if the data was not successfully generated.

4 Glossary and Abbreviations

4.1 Glossary

Term	Description
DaVinci CFG	DaVinci Configurator 5: The BSW and RTE Configuration Editor.

Table 4-1 Glossary

4.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
COM	Communication
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
E2EXF	E2E Transformer
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
LDCOM	Efficient COM for Large Data
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
MPU	Memory Protection Unit
RIP	RTE Implementation Plugin
PPORT	Provide Port
RPORT	Require Port
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 4-2 Abbreviations

5 Additional Copyrights

The MICROSAR Classic RTE Analyzer contains Free and Open Source Software (FOSS). The
following table lists the files which contain this software, the kind and version of the FOSS, the license under which this FOSS is distributed and a reference to a license file which contains the original text of the license terms and conditions. The referenced license files can be found in the directory of the MICROSAR Classic RTE Analyzer.

File	FOSS	License	License Reference
MicrosarRteAnalyzer.exe MicrosarRteAnalyzerCfgGen.exe MicrosarRteAnalyzerCfgGen64.exe MicrosarRteAnalyzer MicrosarRteAnalyzerCfgGen MicrosarRteAnalyzerCfgGen64	Perl 5.30	Artistic License	License_Artistic.txt
MicrosarIRAnalyzer.exe MicrosarIRAnalyzer	llvm 3.6.2 vssa r343 Clang 3.6.2	LLVM License	License_LLVM.txt

Table 5-1 Free and Open Source Software Licenses

6 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com